جزوه‌ی هفته‌ی ۲: کلاس، `__init__` و وراثت

یک رابط مشترک برای همه‌ی حسگرهای وسیله

هوش مصنوعی برای مهندسی هوافضا و مکانیک (دکتری)، بخش صفر

منشأ محتوا: محتوای جدید. جزوه‌های مدرس برنامه‌نویسی شیءگرای پایتون را پوشش نمی‌دهند. مطالب پایتون با بازنویسی (نه رونویسی) از W3Schools (کلاس‌ها، متد `__init__`، ویژگی‌های کلاس، وراثت، چندریختی) و GeeksforGeeks (متغیرهای کلاس و نمونه، `super()`، `property()`، کلاس‌های انتزاعی، dataclass و شروع کار با Pytest) تهیه شده است. مدل ارتفاع فشاری ISA برگرفته از J. D. Anderson, *Introduction to Flight*, Ch. 3 و دستگاه محلی زمین تخت برگرفته از P. D. Groves, *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*, Ch. 2 است.

۱. مسئله

وسیله‌ی آزمایشی ما سه حسگر دارد. **IMU** با نرخ ۵۰ هرتز شش عدد می‌دهد (نیروی ویژه بر حسب $\mathrm{m/s^2}$ و سرعت زاویه‌ای بر حسب deg/s)، بایاس ثابت دارد و در حد بازه‌ی خود اشباع می‌شود. **LiDAR** شانزده‌پرتوی با نرخ ۱۰ هرتز برد می‌دهد و اگر پرتویی بازتاب نگیرد ۰٫۰ یا بی‌نهایت گزارش می‌کند. گیرنده‌ی **GPS** با نرخ ۵ هرتز عرض و طول جغرافیایی، ارتفاع و HDOP می‌دهد و برخی موقعیت‌ها خراب‌اند. همه‌ی بخش‌های بعدی درس (تشخیص خط، تشخیص مانع، کنترل‌گر و پروژه‌ی پایانی) از این داده‌ها تغذیه می‌کنند.

راه رویه‌ای این است که برای هر کار یک تابع بنویسیم (`validate`، `preprocess`، `to_frame`، `plot`) و درون هر کدام زنجیره‌ی `if/elif` روی نوع حسگر بگذاریم. این روش از چهار جهت شکست می‌خورد و همه در آزمایشگاه زنده نشان داده می‌شوند: نوع غلط‌نوشته (`"IMU"` به‌جای `"imu"`) از زنجیره عبور می‌کند و داده‌ی بایاس‌دار را **بدون هیچ خطایی** برمی‌گرداند؛ نمونه‌ای با تعداد مقدار نادرست پذیرفته می‌شود چون `zip` زود متوقف می‌شود؛ پیکربندی‌هایی مثل بایاس به متغیر سراسری می‌روند و داشتن دو IMU با بایاس متفاوت ناممکن می‌شود؛ و افزودن بارومتر یعنی ویرایش همه‌ی توابعی که آن زنجیره را دارند. برنامه‌نویسی شیءگرا هر چهار مشکل را حل می‌کند: داده‌ی هر حسگر را با رفتارش یکجا می‌کند و همه‌ی حسگرها را پشت یک رابط مشترک قرار می‌دهد.

۲. کلاس، شیء، `__init__` و `self`

کلاس نقشه است و **شیء** (نمونه) یک چیز ساخته‌شده از آن. متد ویژه‌ی `__init__` هنگام ساخت شیء خودکار اجرا می‌شود و ویژگی‌ها را روی `self`، یعنی همان شیء در حال ساخت، ذخیره می‌کند. **متد** تابعی است که درون کلاس تعریف شده؛ وقتی `baro.altitude_m(95_000.0)` را صدا می‌زنید، پایتون خود `baro` را به‌عنوان نخستین آرگومان یعنی `self` می‌فرستد.

مثال حل‌شده‌ی ۱: ارتفاع فشاری ISA

در تروپوسفر، اتمسفر استاندارد بین‌المللی فشار استاتیک p را به ارتفاع مربوط می‌کند:

$$h = \frac{T_0}{L}\left[1 - \left(\frac{p}{p_{\mathrm{ref}}}\right)^{RL/g_0}\right], \qquad \frac{T_0}{L} = \frac{288.15}{0.0065} = 44\,330.8\ \mathrm{m}, \qquad \frac{RL}{g_0} = 0.190263.$$

برای $p = 95\,000$ پاسکال و فشار مرجع استاندارد $101\,325$ پاسکال: $p/p_{\text{ref}} = 0.937577$، $\ln(0.937577) = -0.064457$ و $0.190263 \times (-0.064457) = -0.012264$. پس $(p/p_{\text{ref}})^{0.190263} = e^{-0.012264} = 0.987811$ و

$$h = 44\,330.8 \times (1 - 0.987811) = 44\,330.8 \times 0.012189 = 540.3 \text{ m}.$$

کلاس `SimpleBarometer` در `__init__` مقادیر `sensor_id` و `reference_pressure_pa` را ذخیره می‌کند و این رابطه را در یک متد پیاده می‌کند. شیء دومی که با QNH فرودگاه یعنی ۱۰۰٫۵۰۰ پاسکال ساخته شود، برای همان فشار ۴۷۲٫۲ متر برمی‌گرداند: یک کلاس، دو شیء، دو وضعیت متفاوت.

۳. ویژگی کلاس، ویژگی نمونه و property

ویژگی‌ای که در بدنه‌ی کلاس تعریف شود متعلق به **کلاس** است و میان همه‌ی اشیا مشترک است؛ ویژگی‌ای که روی `self` تعریف شود متعلق به **یک شیء** است. اشتراک برای ثابت‌ها درست است (نام کانال‌های همه‌ی IMUها یکسان است) و برای وضعیت نادرست. خطای کلاسیک، تعریف `readings = []` در سطح کلاس است: `self.readings.append(x)` همان لیست مشترک را پیدا و تغییر می‌دهد و هر حسگر نمونه‌های حسگرهای دیگر را هم می‌بیند. در مقابل، انتساب `obj.unit = "rad/s"` یک ویژگی نمونه‌ی تازه می‌سازد که روی ویژگی کلاس *سایه* می‌اندازد. قاعده: **وضعیت تغییرپذیر در `__init__` ساخته شود.**

property شبیه صفت است اما متد اجرا می‌کند. *setter* آن می‌تواند اعتبارسنجی کند: `imu.rate_hz = 0` پیش از آنکه صفری به تقسیم برسد خطای `ValueError` می‌دهد. property بدون setter **فقط‌خواندنی** است (شناسه‌ی حسگر هرگز نباید تغییر کند) و property محاسباتی مثل `period_s = 1 / rate_hz` هرگز کهنه نمی‌شود. خود مقدار در صفتی با زیرخط آغازین (`_rate_hz`) نگه داشته می‌شود که در پایتون یعنی «داخلی».

۴. وراثت، `super()` و ترتیب حل متد

کلاس `class Barometer(Sensor)` همه‌ی متدها و propertyهای `Sensor` را **به ارث می‌برد** و می‌تواند موارد تازه اضافه کند یا موارد موجود را **بازنویسی** (override) کند: همان نام، بدنه‌ی جدید. اگر فرزند `__init__` خودش را لازم داشته باشد، باید اجازه دهد والد بخش خودش را مقداردهی کند: `super().__init__(sensor_id, rate_hz)`. پایتون متد را در امتداد **ترتیب حل متد** (MRO) جست‌وجو می‌کند، مثلاً `Barometer → Sensor → RatedSensor → object`. اگر فرزند `super().__init__` را فراموش کند، ساخت شیء موفق می‌شود اما ویژگی‌های والد هرگز ساخته نمی‌شوند و خطای `AttributeError` خیلی دیرتر و دور از علت اصلی ظاهر می‌شود. مسئله‌ی ۲ کلاسی یک زنجیره‌ی سه‌سطحی را با دست دنبال می‌کند.

۵. کلاس پایه‌ی انتزاعی و dataclass

کلاس پایه‌ی معمولی با یک `preprocess` بی‌اثر اجازه می‌دهد زیرکلاسی که متد را فراموش کرده (یا غلط نوشته) بی‌صدا اجرا شود. **کلاس پایه‌ی انتزاعی** رابط را به قرارداد تبدیل می‌کند: `class BaseSensor(ABC)` متدهای `validate` و `preprocess` را با `@abstractmethod` اعلام می‌کند و پایتون از ساختن خود `BaseSensor` یا هر زیرکلاسی که هر دو را پیاده نکرده باشد خودداری می‌کند. خطا هنگام ساخت شیء و روی میز آزمایش رخ می‌دهد، نه در پرواز.

هر نمونه‌ی ذخیره‌شده یک `SensorReading(timestamp, values, sensor_id, is_valid)` است. اگر این رکورد را دستی بنویسیم به `__init__`، `__repr__` و `__eq__` نیاز دارد؛ `@dataclass` هر سه را از فیلدهای دارای نوع

می‌سازد. `frozen=True` رکورد را تغییرناپذیر (اندازه‌گیری ثبت‌شده نباید تغییر کند) و hashپذیر می‌کند و `slots=True` دیکشنری `__dict__` هر شیء را حذف می‌کند که برای ۱۰۰,۰۰۰ نمونه اهمیت دارد. `values` یک `tuple` است نه آرایه‌ی NumPy، چون `array == array` آرایه برمی‌گرداند و `__eq__` و `hash` ساخته‌شده را خراب می‌کند.

۶. سلسله‌مراتب درس و الگوی متد قالب

فایل `src/aerospace_ai/sensors.py` همه‌ی موارد بالا را ترکیب می‌کند. `BaseSensor` هر آنچه را برای همه‌ی حسگرها یکسان است بر عهده دارد: شناسه، نرخ اعتبارسنجی‌شده، بافر محدود، تبدیل به آرایه و DataFrame، و متد `add_reading` که الگوریتمی ثابت را اجرا می‌کند: بررسی شکل، بررسی افزایش زمان، فراخوانی `validate(raw)`، فراخوانی `preprocess(raw)` در صورت اعتبار، و ذخیره‌ی `SensorReading`. فقط گام‌های ۳ و ۴ را زیرکلاس می‌نویسد. این الگوی **متد قالب** (template method) است. نمونه‌های نامعتبر برای عیب‌یابی به‌صورت خام و با `is_valid=False` ذخیره و از `to_array()` کنار گذاشته می‌شوند.

مثال حل‌شده‌ی ۲: حذف بایاس و اشباع IMU

حد بازه‌ی IMU تنظیم‌شده روی ±۴g برابر $4 \times 9.80665 = 39.23$ m/s² است. کالیبراسیون، بایاس شتاب‌سنج را $(0.10, -0.05, 0.20)$ m/s² و بایاس ژیرو را $(0.8, -0.3, 0.1)$ deg/s داده است.

- نمونه‌ی خام $(0.35, -0.05, 10.05 \mid 0.8, -0.3, 2.1)$ به $(0.25, 0, 9.85 \mid 0, 0, 2.0)$ تبدیل می‌شود و $n = \sqrt{0.25^2 + 9.85^2}/g_0 = 9.8532/9.80665 = 1.005$.
- نمونه‌ی خام $(0.10, \text{NaN}, 9.90 \mid \dots)$ نامعتبر است (متناهی نیست) و خام ذخیره می‌شود.
- مقدار خام $a_z = 39.50 \geq 39.23$ **اشباع** است: حسگر بریده است، مقدار واقعی دست‌کم ۴g و نامعلوم است، پس نمونه نامعتبر است.
- نمونه‌ی خام $(1.30, -0.45, 19.80 \mid 0.8, 4.7, 0.1)$ به $(1.2, -0.4, 19.6 \mid 0, 5.0, 0)$ تبدیل می‌شود و $n = 19.641/9.80665 = 2.003$؛ همان مثال هفته‌ی ۱.

متد `IMUSensor.calibrate` هر دو بایاس را از داده‌ی حالت توقف تخمین می‌زند که در آن نیروی ویژه‌ی واقعی $(0, 0, +g_0)$ و سرعت زاویه‌ای واقعی صفر است. با اعمال آن روی لاگ پرواز هفته‌ی ۱ و بازه‌ی ±۴g، سه خار +۴۰ m/s² به حد بازه می‌رسند و رد می‌شوند؛ بیشینه‌ی ضریب بار نمونه‌های معتبر **۱٫۸۱g** است که زیر حد ۲٫۰g قرار دارد. در هفته‌ی ۱ همین پاسخ به میانه‌ی متحرک نیاز داشت.

۷. چندریختی، حسگرهای تمرین ۱ و آزمون

چندریختی یعنی یک فراخوانی با رفتار مخصوص هر نوع: `for s in suite: s.add_reading(t, raw)` برای هر ترکیبی از حسگرها و بدون `if/elif` روی نوع کار می‌کند، پس `ThermometerSensor` جدید بدون هیچ تغییری در حلقه اضافه می‌شود.

ویژگی **LiDAR** `channel_names` را بازنویسی می‌کند (تعداد کانال‌هایش به `n_beams` بستگی دارد)؛ پرتو i در زاویه‌ی $-180° + i \cdot 22.5°$ است. پرتویی قابل استفاده است که بردش متناهی و در بازه‌ی $[0.15, 12]$ متر باشد؛ پرتوهای غیرقابل استفاده NaN می‌شوند و هرگز صفر نمی‌شوند، چون برد صفر به‌معنای مانعی چسبیده به ربات خوانده می‌شود. اسکنی معتبر است که دست‌کم نیمی از پرتوهایش قابل استفاده باشند.

مثال حل‌شده‌ی ۳: تبدیل GPS به متر محلی

با $R = 6\,371$ کیلومتر، یک درجه عرض جغرافیایی همه‌جا $R\pi/180 = 111\,195$ متر است و یک درجه طول جغرافیایی $111\,195 \cos\varphi_0$. در مهرآباد ($\varphi_0 = 35.6892°$، $\cos\varphi_0 = 0.81221$) این مقدار ۹۰٬۳۱۴ متر است. پس موقعیتی که $0.001°$ شمال و $0.001°$ شرق مبدأ باشد در $(e, n) = (90.3, 111.2)$ متر قرار دارد:

$$n = R\,\Delta\varphi, \qquad e = R\cos\varphi_0\,\Delta\lambda, \qquad u = h - h_0\,.$$

خطای این تقریب زمین تخت در مسیر آزمایشی ۸۰ متری کمتر از ۱ سانتی‌متر است. موقعیت‌هایی با HDOP بزرگ‌تر از ۵، مختصات NaN یا $|\text{lat}| > 90°$ رد می‌شوند.

آزمون. آزمون `pytest` یک تابع `test_*` با `assert` ساده است. `pytest.raises` بررسی می‌کند که خطا رخ دهد، `@pytest.mark.parametrize` یک آزمون را روی ورودی‌های متعدد اجرا می‌کند و `@pytest.fixture` برای هر آزمون یک شیء تازه می‌سازد. آزمون خوب **عدد** را بررسی می‌کند: تمرین ۱ مجموعه‌ی آزمون هر دانشجو را روی سه پیاده‌سازی با خطای ظریف اجرا می‌کند (شرق بدون $\cos\varphi_0$ ، نادیده‌گرفتن حد HDOP، نگه‌داشتن بازتاب‌های خارج از برد) و برای هر کدام که شکار شود امتیاز می‌دهد.

آنچه اکنون بلدیم انجام دهیم

- داده و رفتار را در کلاس جمع کنیم و از `__init__` و `self` درست استفاده کنیم.
- وضعیت تغییرپذیر را برای هر شیء جدا نگه داریم و صفت‌ها را با property اعتبارسنج و فقط‌خواندنی محافظت کنیم.
- با وراثت و `super()` کد را بازاستفاده و با کلاس پایه‌ی انتزاعی رابط را اجباری کنیم.
- رکورد تغییرناپذیر را با `@dataclass(frozen=True, slots=True)` بنویسیم.
- فهرستی ترکیبی از حسگرها را چندریختی پردازش و سلسله‌مراتب را با pytest آزمون کنیم.

تمرین: مسائل کلاسی `in_class_problems/week02_problems.ipynb`، آزمایشگاه `notebooks/week02/week02_oop_sensors.ipynb` و تمرین ۱ (`assignments/assignment01/`).